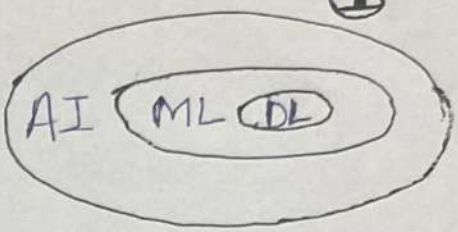
MACHINE LEARNING



AI Vs ML Vs DL

→ AI = System that mimics human intelligence.
→ ML = branch of CS that learn patterns from data, instead of rules
→ DL = Uses neural networks with many layers to learn features automatically.

- In ML, you tell the model what to look at (features), and it learns how to decide eg cat or dog.
- In DL, you give raw data, and the model learns what to look at and how to decide by itself.

Types of Machine learning

- Supervised
  - Regression
  - Classification
- Unsupervised
  - Clustering
  - Dimensionality Reduction
  - Anomaly detection
  - Association Rule learning
- Semisupervised
- Reinforcement

1) Supervised

You show the model Eg:-

> Input + right output column and it learns the pattern

Input		Output
IQ	CGPA	Placement
87	7.1	Yes
111	8.9	No
75	6.3	No

→ Types of supervised ML:-

- Regression = Used when output/target column has Numerical values [eg: 15, 2.5, 0]
- Classification = Used when output/target column has Categorical values [eg: Yes, No]

2) Unsupervised

You show the model:-

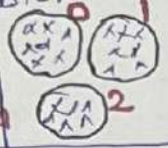
Only the ~~model~~ input data and it find patterns itself

Input	
IQ	CGPA
70	10
60	6
100	9

→ Types:-

Clustering

Groups similar data points together as clusters



eg:- 0 → high IQ, CGPA
1 → high IQ, low CGPA
2 → low IQ, low CGPA

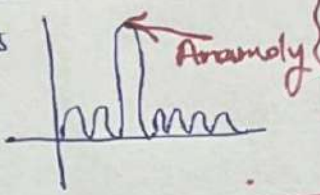
Dimensionality Reduction

Reduces many features to fewer meaningful ones

eg - Compress 100 features into 2-3 for visualization

Anomaly Detection

Find data points that look very different from the rest



Association Rule mining

Find items/events that occur together

eg - People often buy bread with butter

3) Semi Supervised

Note → Output column is also called label

→ Semi supervised is partially supervised & partly unsupervised.

→ A small amount of data is labelled by humans, and large amount is unlabeled. The model uses both to learn.

eg: 10000 emails → 500 labelled by humans (spam/not spam) [Supervised] + 9,500 email have no labels [Unsupervised]

Model learns from the 500 labels email & use that pattern in 9500 unlabeled

4) Reinforcement Learning

When a model learns by trial and error, getting rewards or penalties for its actions

eg- Self driving Car
- Action: Accelerate / Brake
- Reward: Stay in lane
- Penalty: collision, sudden brake

*Learns safe driving by trial & feedback.

Batch Vs Online ML

These describe how a ML model is trained and updated over time in production setup.

1) Batch Learning [Offline Learning]

→ Model is trained on full dataset at once locally & then deployed.
→ It does not update until retrained locally & deployed again.

Disadvantage:-
- Lots of data
- Hardware limitation
- Availability [In terms of time/eg news or quick update]

2) Online Learning

→ Model updates itself continuously in production as new data comes in.
→ ~~Tools - eg scikit learn, Pytorch~~.

★ Usage:-
- When there is a concept drift
- Cost Effective
- Faster Solution

} As chunks (small) are processed in online learning

★ Challenge = Learning Rate:-
- The rate at which the online model learns
- To set the correct learning rate is a challenge in Business, to remember the old stuff and learn new stuff.

★ Out of Core learning:-
- Used when data is too large to fit the memory, so the model is trained in chunks instead of all at once.
- It's about memory only, can be used in both online or offline ML

★ Disadvantage:-
- Tricky to Use
- Risky

Instance Based Vs Model Based Learning

Describes how a model learns — by remembering data directly or by learning a general rule from it.

1) **Instance Based**

Makes prediction by comparing new data to stored training examples. Idea is to "Remember & Compare"

eg: K-Nearest Neighbours (KNN):-
A new points is classified based on the labels of nearest points

2) **Model Based**

Learns (rules/parameters) from data & uses it for prediction.

eg:- Linear Regression, Decision Tree etc.

Challenges in Machine Learning

1) Data Collection
2) Insufficient Data/Labelled Data
3) Non Representative Data
4) Poor Quality Data
5) Insufficient features
6) Overfitting
7) Underfitting
8) Software Integration
9) Deployment/Batch learning
10) Cost Involved

Application of Machine Learning

1) Retail - Amazon/Big Bazar
2) Banking & Finance
3) Transport - OLA
4) Manufacturing Tesla
5) Consumer Internet - Twitter.

Machine Learning Development Life Cycle (MLDLC)

Frame the Problem
↓
Gathering the Data
↓
Data Preprocessing
↓
Exploratory Data Analysis
↓
Feature Engineering & selection
↓
Model training, Evaluation & Selection
↓
Model Deployment
↓
Testing
↓
Optimize

→ Standard ML Process/cycle.

What are Tensors? [n-dimensional Array]

→ A Tensor is a container for numbers with structure.
→ It doesn't just hold numbers, it also knows how those numbers are arranged.

★ 0D Tensor :-

(2) or (3) — Scalars

This has 0 dimension.

★ 1D Tensor :-

Combination of multiple 0D Tensors {Scaler Tensors}

eg [1,2,3,4]

Also called 1D array / Vector
Note = But the dimension of vector will be 1 Dimension not 4.

★ 2D Tensor

Combination of multiple 1-D Tensors

$$\begin{bmatrix} [1,2,3] \\ [4,5,6] \\ [7,8,9] \end{bmatrix} \rightarrow \text{Dimension} = 2$$

★ ND Tensor

A tensor with n dimensions

★ Rank, Axes & Shape

No. of axis = No. of dimensions = Rank

Shape = (Rows, Columns) = (2,3) here $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

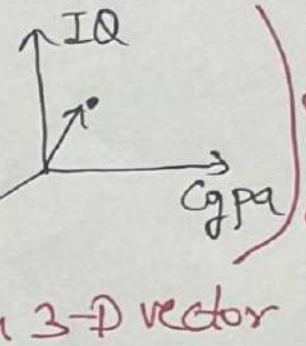
Examples

1-D Tensor

(eg)

cgpa	iq	~~Age~~	placement
8.1	91	21	1

[8.1, 91, 21] ⇒



1-D Tensor but represented in 3-D vector

2-D Tensor

$$\begin{bmatrix} [\text{student 1 data}] \\ [\text{student 2 data}] \\ \vdots \end{bmatrix} \quad \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Input [2-D Tensor] Output [1-D Tensor]

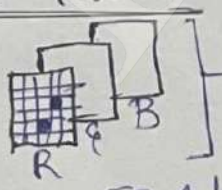
3-D Tensor

$$\begin{bmatrix} [[1,0,0,0],[0,1,0,0]] \\ [[1,0,0,0],[1,0,1,0]] \\ [[0,0,0,1],[0,0,1,0]] \\ \vdots \end{bmatrix}$$

** Shape = (3,2,4)

Time series data is also an example of 3-D Tensor eg (10,365,2)

4-D Tensor



→ (3, 1200, 800)

Suppose → 50 color photos

So shape = (50, 3, 1200, 800)

5-D Tensor

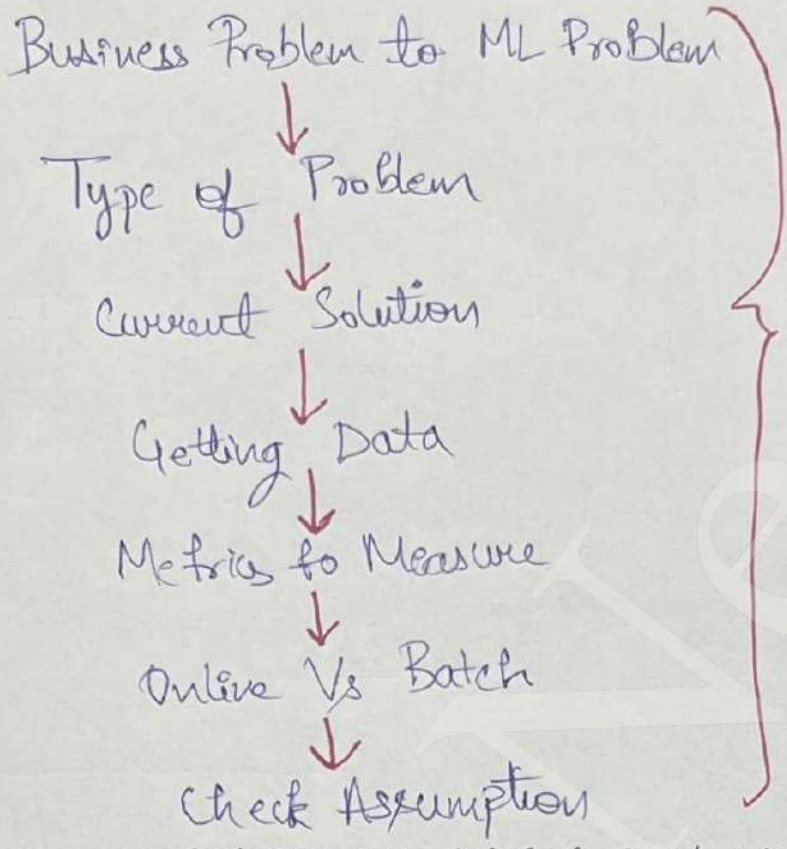
~~Videos = collection of Images~~

Is a collection of videos

(eg) 4 videos of something

(4, 1800, 480, 720, 3)

How to frame a Machine learning Problem



DATA GATHERING

CSV/Excel JSON/SQL Fetching from API Web Scrapi

→ CSV/Excel:-
Used when data is already available as files

→ JSON:-
Used for semi-structured data, mostly from APIs

→ SQL:-
Used when data is stored in databases

→ APIs:-
Used to fetch real time or live data

→ Web Scraping:-
Used only when no API is available.

Understanding your data

Asking basic questions | EDA (Univariate) | EDA (multivariate) | Pandas Profiling

1) Asking basic questions :-

- How big is the data ? = df.shape
- How does the data look like ? = df.head()
- What is the data type of cols ? = df.info()
- Are there any missing values ? = df.isnull().sum()
- How does the data look mathematically ? = df.describe()
- Are there duplicate values ? = df.duplicated().sum()
- How is the correlation between columns ? = df.corr()

2) EDA (Univariate)

→ One variable at a time. Basically how does this single column look like

→ Check: Distribution, Central Tendency, Outlier, Missing values.

→ Common visuals: Histogram, Box plot.

3) EDA (Bivariate)

→ Two variables together.

→ How one column change with another ?

→ Check: Correlation, Trends

→ Common Visuals: Scatter plot

Multivariate

→ More than 2 variable

→ How do multiple columns interact together

→ Check: Combined effects, patterns

→ Common Visuals: Heatmaps, pair plots

4) Pandas Profiling

→ An automated EDA tool that summarizes the dataset for you

→ Helps you quickly understand structure & quality of data.

→ Useful for initial exploration, not final conclusions.

What is Feature Engineering?

→ Feature Engineering is the process of using domain knowledge to extract features from raw data.
→ These features can be used to improve the performance of machine learning algorithms.

FEATURE ENGINEERING

- Feature Transformation
  - Missing Value Imputation
  - Handling Categorical Features
  - Outlier Detection
  - Feature Scaling
- Feature Construction
- Feature Selection
- Feature Extraction

1) Feature Transformation

- Missing Value Imputation – Either remove or fill missing values. Replace by mean, median or most frequent category.
- Handling Categorical features – eg One-hot Encoding
- Outlier Detection – Outliers removed are much better fit.
- Feature Scaling – (eg) Age in range of 100s, Salary in range of 10000s

→ So for KNN eg – $\sqrt{(x_2-x_1)^2+(y^2-y_1^2)}$ → Salary term dominates.
→ Thus we bring all values to common scale.
→ Generally bw −1 to 1

2) Feature Construction

→ Create new column manually using domain knowledge
→ Eg new column by adding 2 columns.

3) Feature Selection

→ Selecting the features useful for the model
→ Speed of the model increases.

4) Feature Extraction

→ Eg - PCA (Principal Component Analysis), LDA
→ this is done by system, not manually.

Feature Scaling

→ It is generally the last thing that we do in ~~FE~~ pipeline.
→ Right before feeding the data to LLM.
→ It is a technique to standardize the independent features present in the data in fixed range.
→ We use FS to prevent a feature from dominating - Eg.

Feature Scaling

Standardization | Normalization

1) Standardization ⭐⭐ [You fit on TRAIN data & apply on TEST data]

→ Also called Z-score Normalization
→ Formula → $X_i' = \frac{X_i - \bar{X}}{\sigma}$, $\text{Mean}(\mu) = 0, \text{Std}(\sigma) = 1$ → This is the range of the scaled column

X_i' : scaled column values
X_i : original column values
$\bar{X}$: mean
σ : std deviation

→ Used in ~~distance~~ Gradient based algorithms, not tree based
→ Centers data at 0 & spread by standard deviation

Note: ⭐⭐
→ Standard deviation tells you the spread of data from average
→ Small std dev → values are close to mean
→ Large std dev → values are spread far apart. } Magnitude

2) Normalization

- **Min-Max Scaling:-**

→ Used when data has fixed bounds & no outlier.

→ Basically known bounds eg min-max-(0,255) for image pixel.

→ Formula: $$X_{scaled} = \frac{X_i - X_{min}}{X_{max} - X_{min}}$$

Used when we know the upper & lower bound

- **Mean Normalization :-**

→ Formula: $$X_{scaled} = \frac{X - X_{mean}}{X_{max} - X_{min}}$$

→ Centers data around 0 by substracting the mean.

- **Max-Abs Scaling :-**

→ Divides each value by maximum absolute value.

→ Formula: $$X_{scaled} = \frac{X_i}{|X_{max}|}$$

Used in case of sparse data

- **Robust Scaling :-**

→ Formula: $$X_{scaled} = \frac{X_i - X_{median}}{IQR}$$

Use in case of outliers

Standardization Vs Normalization

→ Standardization is Used → for gradient based Models (Linear, Logistic, SVM)

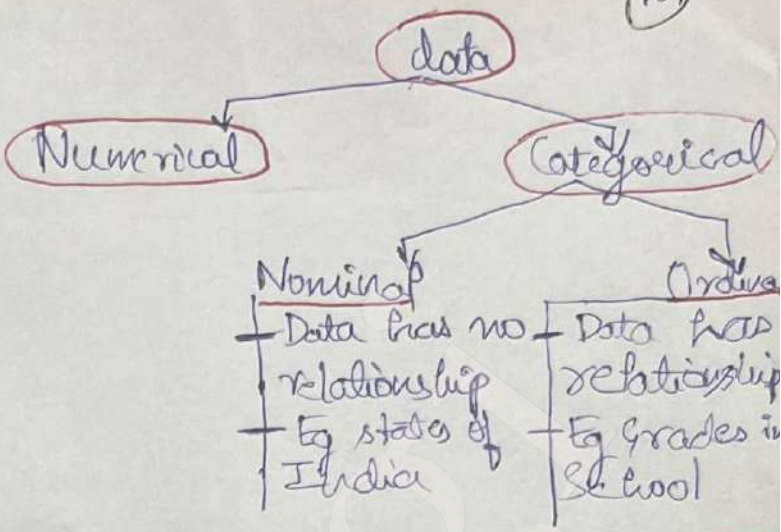
→ Normalization is Used → for distance based Models. (KNN, K-Means)

→ Neither is Used → for tree based Models (Decision tree, Random forest, XGBoost)

Encoding Categorical Data

→ Convert Categorical columns to numbers for ML Algos.

→ Techniques:-
1) Ordinal Encoding
2) Label Encoding
3) One hot Encoding



Ordinal Encoding & Label Encoding

→ Both used in Ordinal Data

(eg) Education

HS	0
UG	1
PG ⟹	2
PG	2
UG	1
HS	0

∵ PG > UG > HS
PG = 2
UG = 1
HS = 0

- Both work exactly same
- Ordinal Encoding: Works on INPUT columns
- Label Encoding: Works on TARGET column

* All transformations fit on train data only & transform both train & test.

3) One hot Encoding

→ Used for Nominal Data

Color	Target
Y	0
Y	1
B	1
Y	1
R	0

OHE →

color_Y	Color_B	Color_R	Target
1	0	0	0
1	0	0	1
0	1	0	1
1	0	0	1
0	0	1	0

** If column has 500 categories then 500 new columns

* Dummy variable Trap = If we have n columns, we use (n-1) to avoid multicollinearity. Even after removing 1 column all row are unique

* OHE using most freq variable - Eg Keeping only the top frequent categories, this will reduce the dimensionality.

Column Transformer

→ In scikit learn column transformer applies different transformations to different column is single pipeline.

→ In real data:
Numerical column – Needs scaling/Imputation
Categorical column – need encoding
some columns – Need to be dropped
} Column Transformer solves this.

→ It combines all transformed output to one single mix.

Machine Learning Pipeline [Raw data → preprocess → Model training → Predict]

→ A ML pipeline is a way to chain all preprocessing steps and model in a single workflow.

→ Ensures same steps are performed during training, ~~testing~~ validation, and inference [testing model on real world data].

→ One .fit() handles everything.

→ Syntax:

```
pipeline = Pipeline([
        ('step1': transformer1),
        ('step2': transformer2),
        ('model': estimator)
        ])
```

→ Any step can be swapped or tuned without changing the downstream code.

→ Pipelines integrate directly with grid search CV/Randomized Search CV.

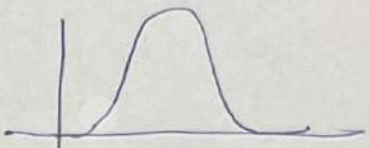
→ Pipelines Reduce manual errors & allow end to end evaluation.

Function Transformer

→ Allows us to use custom feature transformation inside the pipeline

→ Function here means a rule or data eg log(x), X/10000.

Not - Normal Distribution



- Values cluster around mean
- Fewer value away from mean

Scikit-learn Transformers

- Function Transformer
  - Log Transform
  - Reciprocal Transform
  - Sq / Square Root
  - Custom
- Power Transformer
  - Box-Cox
  - Yeo-Johnson
- Quantile Transformer

Note - Many ML algo work best & assume features are normally distributed

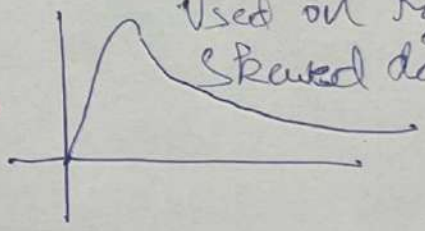
1) Function Transformer:

* Log Transform -

→ Age | log(Age)
21 →
22

} The data becomes more normally distributed

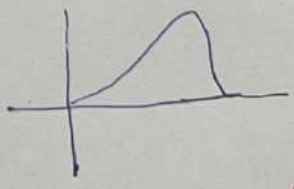
→ Not applied on -Ve values.



Used on Right Skewed data.

* Reciprocal transform =
$\frac{1}{X}$ = Reciprocal compresses large values aggressively.

* Square transform
$(X)^2$ = Works best on left skewed data



* Square root transform
$\sqrt{x}$ = Count or mildly skewed data, especially when 0 are present

* Custom - Any custom logic.

Power Transformer

→ Adaptive transformer that learn λ from data
In function transformer we choose λ here it learns

Box-Cox Transform

→ All values are strictly positive & $X > 0$
→ Most normal possible distribution with best λ
Range = $(-5, 5)$

$$x_i^{(\lambda)} = \begin{cases} \dfrac{x_i^{\lambda} - 1}{\lambda}, & \lambda \neq 0, \\ \ln(x_i), & \lambda = 0, \end{cases}$$

Yeo-Johnson

→ Extension of Box-Cox Transform
→ Works for negative and zero unlike ~~Box~~ Box-Cox Transformer.

Binning & Binarization

→ These are data preprocessing techniques used to transform continuous numerical feature into categorical interval.
→ Helps model capture meaningful patterns.

Binning (Discretization) → Handles outliers

→ Interval.

→ Converts a continuous numerical feature into categorical (bins).
→ Need - few real world behaviour happens in ranges.
→ eg) Salary Ranges (Low, Med, High) instead of exact values.

Types of Binning :-

- Unsupervised
  - Equal Width (Uniform)
  - Equal Frequency (Quantile)
  - Kmeans Binning
- Supervised
  - Decision Tree Binning
- Custom Binning

* Equal Width / Uniform Binning

→ You close the bin count, eg Bins = 10

→ Formula: $\frac{max - min}{bins}$

eg: max = 100, min = 0
Bins = 10
↓
10 Bins of interval 10 each will be made

Age
27
32
84
⋮

* Equal Binning

Result :

age	age-labels
27	(20, 30]
32	(30, 40]
84	(80, 90]

→ No change in spread of data but handle outliers.

* Equal Frequency / Quantile Binning

→ Suppose Bins = 10 → percentile

→ Each interval will contain 10% of total Observation

→ Intervals: 0-16, 16-20, 20-22

10% percentile population reside here

0-20, 20% percentile population lies

0-32, 30% percentile population lies

- The binning size is not equal.
- This is used more

* K-Means Binning

→ Used when data is spread in clusters.

→ Clustering is done & each bin act as a cluster.

How Binning works in Scikit-learn

KBinsDiscretizer()

Parameters used

- Bins = ?
- Strategy
  - Uniform
  - Quantile
  - Kmeans
- Encoding *
  - Ordinal
  - Numerical

* Custom binning

→ Use own knowledge or domain knowledge for binning.
→ Can't be done using scikit learn, use pandas custom code

Binarization

→ Converts a numerical feature into binary (0/1) using threshold
→ Eg: Income > 50,000 → 1 else 0

Handling Mixed Variable / Data

→ Column has mixed data [Numerical + Categorical]

Case 1

Cabin
B5
C23
D41

→

B	5
C	23
D	41

Case 2

7
3
1
AB
4

→

7	NA
3	NA
1	NA
NA	AB
4	NA

Handling Date & Time Variables

→ Date & time often encode patterns like seasonality, cycles & trends
→ Never use date directly, extract date, time etc or use sin/cos, log etc
→ Convert to datetime ⇒ df["date"] = pd.to_datetime(df["date"]);

Handling Missing Data

- Remove them
- Impute
  - Univariate
    - Numerical
      - Mean/Median
      - Random
      - End of distribution
    - Categorical
      - Mode
      - "Missing"
  - Multivariate
    - KNN imputer
    - Iterative Imputer MICE

1) Remove them / Complete case Analysis

→ Also known as list-wise deletion.

→ It means discarding observations where values in any of the row is missing (observations → complete row)

→ CCC means analyzing only those rows in which we have value in each column.

→ Assumption - Removed data was random, does not hold value. only when <5% values are removed.

Handling missing Numerical Data

→ Impute Missing value with mean / median } Used when less missing values.
(mean → use when data normal; median → skewed Result)

→ Disadvantag - Distribution changes, creates outliers.

→ Arbitrary value Imputation - Used in categorical data mostly

- Impute with word 'Missing'
- Impute with ~~Hot~~ '0' or '1'.
- Impute with something that already does not exist.

→ End of Distribution Imputation -

Normal data: Impute with [mean + 3σ, mean − 3σ]

Skewed data: [Q_1 − 1.5 IQR, Q_3 + 1.5 IQR]

OR Use GridSearchCV, it finds the best params (techniques)

- We impute with the end of values.
- Used when data is not missing at random.

→ Random Sample Imputation :-

- Impute with a random number from existing data
- Keeps distribution Intact

Handling Missing Categorical data

→ Fill by most frequent value. } Mean or Median not possible.
eg fill by Mode = "Mumbai"

→ Missing category Imputation :- If >10% missing
Create a new category, Replace NA with "Missing" = New Categor

KNN Imputer (Multivariate Imputation)

→ Other rows are also used to impute the values.
→ Works on the KNN Algorithm. (Characteristics similar to neighbour
→ Replace the value with the nearest neighbours.
→ More accurate but more time taking.

MICE (Multivariate Imputation By Chained Equations)

→ Missing Completely at random - Data not gathered from source
Missing at random - People didn't fill age somewhere or name
Missing at not random - Data was removed on purpose.
→ MICE Works best for (MAR) and is more accurate

Steps :-
1) Fill all NAN with mean/median
2) Pick one column at a time, treat it as target (Y)
3) Use other columns as ~~target~~ Input (X)
4) Train a model to predict missing values
5) Replace the picked column value with predicted
6) Do this for all columns
7) Do 1-6 step till diff b/w imputed & predicted value is near to 0.

} Iteratively fills values using the other features, instead of simple averages

What are Outliers?

→ A data point different from all other points

→ Few cases outlier need not be removed - eg for anamoly detection problems.

→ Linear Reg, Log Reg, Adaboost, DeepLearning - All are weight based algos, so outliers have high impact here.

→ Treating Outliers

- Trimming (Remove them)
  - pro = fast
  - con = Reduce data size
- Capping ★★
  As outlier is always on Left or Right side, save cap & say anything outside those caps on each sides have value of that cap.

→ Techniques for outlier detection and removal.

- Z-score treatment
- IQR based filtering
- Percentile
- Winsorization

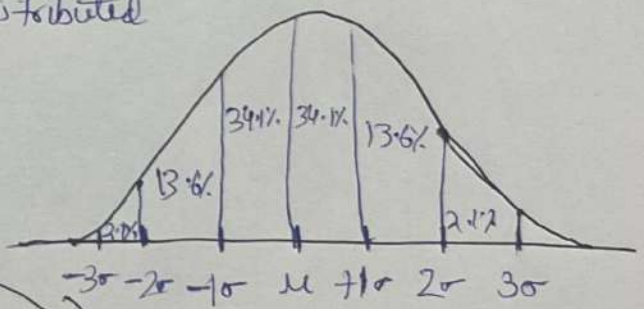
---

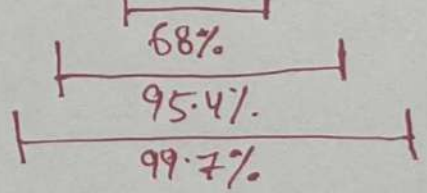
1) Z-score treatment

→ Assumption - the column is normally distributed

This means that:-

- 68% data lies b/w $\{\mu+\sigma, \mu-\sigma\}$
- 95% data lies b/w $\{\mu+2\sigma, \mu-2\sigma\}$





So Anything beyond $\{\mu+3\sigma, \mu-3\sigma\}$ is an outlier.

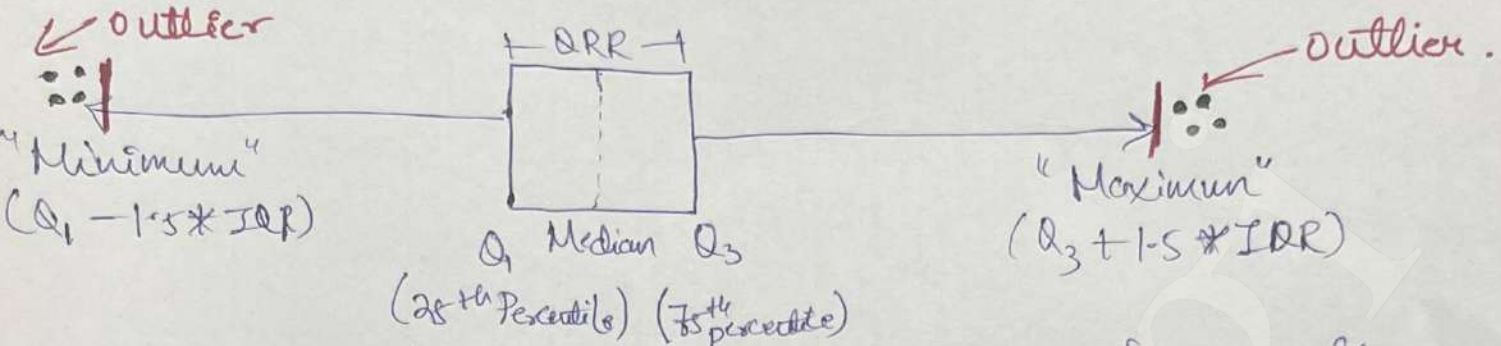
Upper limit

Lower limit

→ This is derived from Z-score formula $= x_i' = \frac{x_i - \mu}{\sigma}$

2) IQR format

→ Used when data distribution is skewed.



→ Anything outside minimum or maximum is an outlier.

3) Percentile

→ eg - 99% means 99% people are behind me
→ If maximum value is 95 - then 100% (percentile)
→ 50 percentile is the median.

** Decide a min & max percentile threshold.

→ Capping using Percentile is called winsorization

Feature Construction

→ Intuition / Domain based manual feature construction
→ Derive a new column with the help of ~~new~~ other columns
→ It might increase models performance.
→ Feature splitting - split Mr Ankit column to 2 columns.

Feature Extraction (Curse of Dimensionality)

→ features → dimensions, basically curse of features.
→ Optimal no. of features, if we add more model would perform poor.
→ With increase of more features sparsity b/w feature increases, not much rel^n stays b/w each features & computation Increases
→ Solution = Dimensionality Reduction → Feature Selection
→ Feature Extraction [PCA, LDA, t-SNE]

Feature Extraction (PCA)

→ Unsupervised technique of feature Extraction.

→ To remove curse of dimensionality, reduce dimension.

→ PCA is a technique which can transform a higher dimension data to a lower dimension data while keeping the essence of the data. ★★

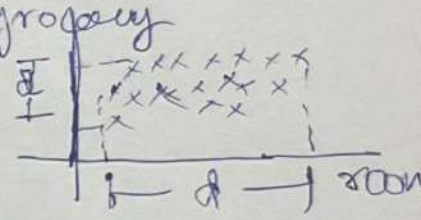
eg - Cameraman captures (2D) image of (3D) stadium at best angle.

→ Visualization becomes easier as dimensions reduces.

Intuition

★ When Feature selection does not help we do feature Extraction (PCA)

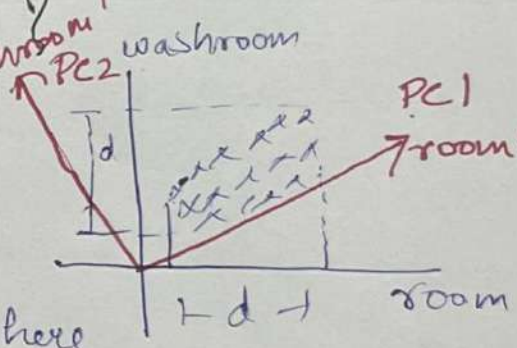
Normally we can simply select with domain knowledge but if we don't have domain knowledge we select feature with [data spread (Variance)]



d > d', so we chose room as the data spread/projection/variance is more here

→ But what if d ≃ d' [Almost equal] ?

- In this case we use feature Extraction (PCA)
- Created PC_1 & PC_2, but we will [Chose PC_1 = room'] as variance is higher here no.



- no of PC ≤ n(features)
- Goal is to have maximum variance while reducing dimension to keep the data points relationship intact.

→ PCA finds the direction (Vector) onto which projecting the data gives the maximum variance, this is how we chose the principal components (basically the new columns extracted) ★★

→ All PC's are orthogonal (Perpendicular) to each other.

Simple Linear Regression

→ Supervised ML algorithm

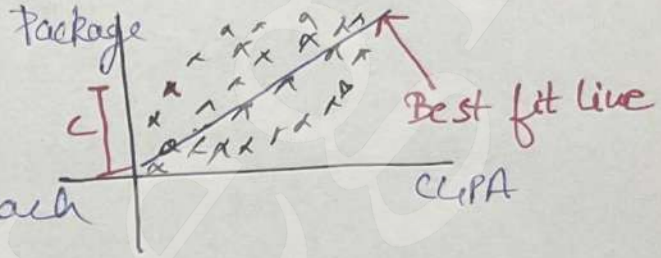
→ Type of Linear Regression:-

- Simple Linear Regression - 1 input column, 1 output column
- Multiple Linear Regression - More than 1 input column & 1 output.
- Polynomial Linear Regression - When data is not linear.

1) Simple Linear Regression:

→ It finds the best fit line

→ With lowest possible error from each point



Equation: $y = mx + c$

y: prediction, m = slope
x: input point, c = y-intercept
dist. of 'x' in y-axis

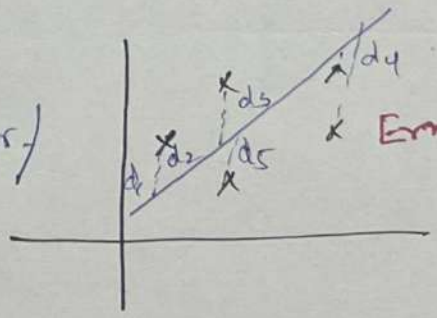
→ Goal is to find optimal 'm' & 'c'.

→ Two techniques to find optimal 'm' & 'c'

- Ordinary Least Square (OLS) (Use when ↓ dimension data)
- Gradient Descent (Use when ↑ dimension)

- Ordinary least Square:-

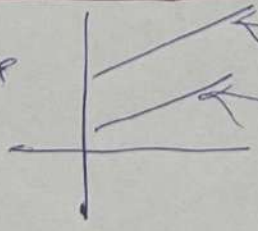
→ The goal is to reduce the error/ distance of point from line



$$E(m, c) = \sum_{i=1}^{n} (y_i - mx_i - c)^2$$

→ optimal c, m for lowest E.

→ Note c = offset error



Regression Metrics

→ MAE, MSE, RMSE – Predicts how wrong the model is in absolute terms (error size in units)

→ R^2 & Adjusted R^2 – Measures or tell how good the model is, explains the variation in Data.

→ MAPE WMAPE – Measures how wrong the prediction is in business term for business.

MAE (Mean Absolute Error)

$$MAE = \frac{\sum_{i=1}^{n} |y_i - \hat{y}_i|}{n}$$

y_i – Actual point value
$\hat{y}_i$ – Predicted point value
n – total data points

Adv: • Have Error in same unit
• Robust to outliers

Disadv: Uses sigmoid fn, so can't be differentiated

MSE (Mean Squared Error)

$$MSE = \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{n}$$

Uses square fn, Differentiable

→ Can be used as loss fn,
→ Disadv • Can't handle outlier well
• Diff unit from target

RMSE (Root Mean Squared Error)

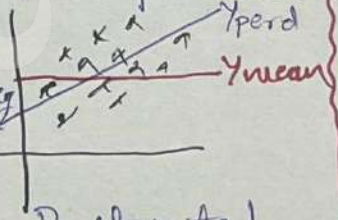
$$RMSE = \sqrt{MSE}$$

→ Same properties of MSE
→ But has same unit from target

R2 (score) / Coefficient of Determination

→ Essentially tells by how much the predicted line deviates from mean line

$R_2 = 1 - \frac{SS_R \text{ (Sum of square Error Reg)}}{SS_{mean} \text{ (Sum of square Error Mean)}}$



$$R_2 = 1 - \frac{\left[\sum_{i=1}^{n} (y_i - \hat{y}_i)^2\right]_{Reg}}{\left[\sum_{i=1}^{n} (y_i - \hat{y}_i)^2\right]_{mn}}$$

• R_2 close to 1 means good model performance

Adjusted R_2

→ R^2 always increases when we add new features, even if new features add no value

→ Adjusted R^2 penalizes extra features & increase

→ Only when a new variable truly improves the model.

$$R^2_{adj} = \left[\frac{(1 - R^2)(n-1)}{(n-1-K)}\right]$$

n → no. of rows
K → independent column count basically number of predictors.

MAPE

→ Mean Absolute % Error

$$MAPE = \frac{1}{n} \sum \left|\frac{Actual - Pred}{Actual}\right| \times 100$$

→ On average how much % the prediction is wrong

WMAPE (Weighted MAPE)

$$WMAPE = \frac{\sum |Actual - Pred|}{\sum Actual} \times 100$$

→ Error in prediction % for business.
→ Lower MAPE or WMAPE is good

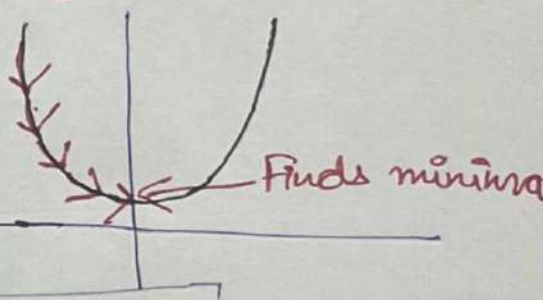
Multiple Linear Regression

→ Input has more than one columns.
→ Instead of best fit line here best fit plane or hyperplane
→ $y = mx + c \longrightarrow y = m x_1 + m x_2 + c \longrightarrow \boxed{y = \beta_0 + \beta_1 X_1 + \beta_2 X_2}$

for n columns $\boxed{y = \beta_0 + \sum_{i=1}^{n} \beta_i x_i}$ → Goal is to find lowest/optimum values of offset & slope.

Gradient Descent

→ In higher dimension the old method to find m, & c fails.
→ helps to find Minima of a differentiable function.
→ Used in many algorithms and Deep learning.
→ Gradient descent is an optimization algorithm used to minimize the loss (error) of machine learning model.
→ It adjust the model parameters step by step to reduce prediction error.
→ The step size is the learning rate (η)
→ Gradient descent slowly adjust m & c to reduce prediction error step by step.



$$\boxed{\text{New value} = \text{Old value} - \text{Step size}(\eta) \times \text{slope}}$$

Steps
1) Start with random m, c = m=5, c=10
2) Pred output, y-pred = m.x+c, Calculate Error = actual - pred.
3) Find slope/Gradient by differentiating
4) How loss changes by changing 'm' or 'c', based on that update m & c.
5) m_new = m - learning rate × (slope w.r.t m)
   c_new = c - learning rate × (slope w.r.t c)
6) Repeat these until error is not reduced anymore & slope is almost 0
7) Thus we find best 'm' & 'c'.

Types of Gradient Descent

Batch GD | Stochastic GD | Mini batch GD

1) Batch Gradient Descent

→ Uses entire dataset to calculate gradient
→ One single update after seeing all data

Very slow & need lot of memory

2) Stochastic Gradient Descent

→ Uses one data point at a time
→ Updates 'm' & 'c' after every record
→ 300 rows data means 300 time update

Work fast
Used for Big data
Here loss fluctuates a lot

3) Mini Batch Gradient Descent

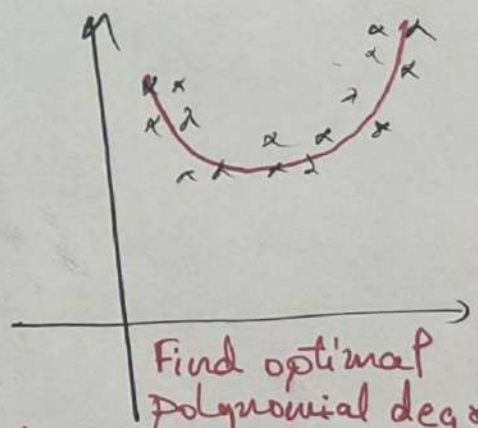
→ Use small batch of data (like 32, 64 rows)
→ Most widely used.
→ For each batch :-
- Compute loss
- Compute Gradient
- Update 'm' & c

} Repeat for each batch & then for each epoch (iteration)

FAST
Stable
Efficient Memory Use

Polynomial Regression (Coefficients are still linear, thus called Linear Reg)

→ Used when ~~linear~~ relationship b/w x & y is not a straight line
it is a curve



→ We add a Polynomial degree to linear Eq^n

$y = mx + c \rightarrow y = \beta_1 X + \beta_0$

(e.g.) suppose

X	Y
1	1
2	0
3	0

, so polynomial degree = 2

$y = \beta_0 + \beta_1 x + \beta_2 x^2$

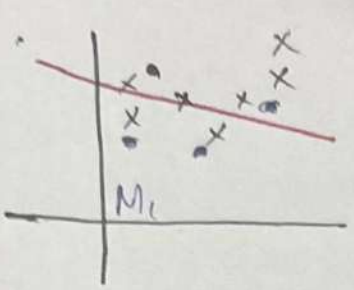
FINDS BEST FIT CURVE

→ Similarly degree 3 = $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$

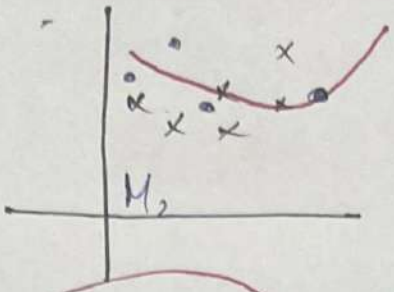
Find optimal polynomial degree to avoid overfitting or underfitting

→ Used when data is non-linear (curved) & straight line does not fit well.

Bias-Variance Tradeoff

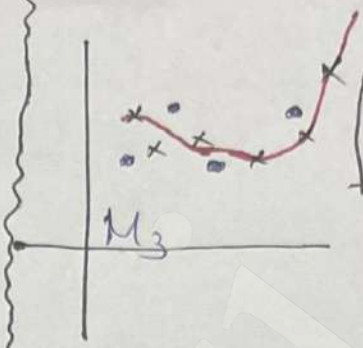


High Bias
Low Variance
[Underfitting]



Low Bias
Low Variance

Bias Variance Tradeoff
The sweetspot.



Low Bias
High Variance.
[Overfitting]

3 ways to achieve Bias-Variance Tradeoff:-

1) Regularization
2) Bagging
3) Boosting

Note:

Bias	Variance
• Model is too simple	• Model is too complex
• Pattern Dikha hi ni	• Har point pe vishwas

1) Regularization (Ridge Regression)

→ Used to reduced overfitting.

Types

- L2 Regularization (Ridge)
- L1 Regularization (lasso)
- Elastic Net.

• (L2) Ridge Regression

→ Ridge regression reduces overfitting by shrinking coefficients, not removing them.

(eg) Loss = Loss + Penalty (Add squared penalty on coefficients)

→ Shrinks coefficients but never make them zero.

→ Used in cases where all features are important.

(L1) Lasso Regression

→ Lasso Regression reduces overfitting by shrinking weights, some weights become exactly zero.

→ Thus effectively it does feature selection

→ Loss = Loss + Penalty (Add absolute penalty on coefficients)

$$L = Loss + \lambda |m|$$ $\lambda \geq 0$, Higher lambda – Underfit, Lower lambda – Overfitting.

→ Why Lasso Creates sparsity:-

- As λ increases so the corresponding coefficients become zero. For ridge near to 0, for lasso exactly 0.

If feel like λ is in numerator in lasso & denominator in Ridge

---

Elastic net Regression

→ Combination of Ridge & Lasso Regression

→ If dataset huge & confused to use L_1 or L_2 regularization

$$Loss = Loss + a|w|^2 + b|w|$$ (Ridge, Lasso) a-controls shrinking coeff, b-controls feature remove.

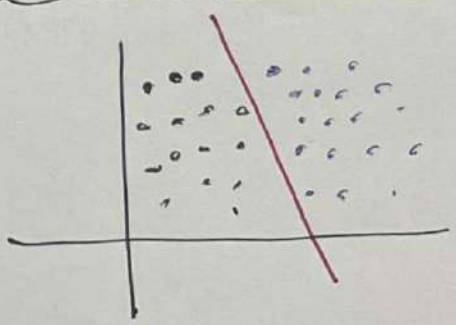
Note : Correlation eg = Height & Weight

Collinearity eg = duplicate, eg house in sqft or house in sqm.

multicollinearity eg = similar kind multiple columns. one can be derived from others (duplicate basically)

→ Works well when input column has high multicollinearity.

Logistic Regression



→ Data should be linearly seperable.

→ We use general eq^n of line here

$y = mx + b \rightarrow AX + By + C = 0$

→ Goal is to find A, B, C

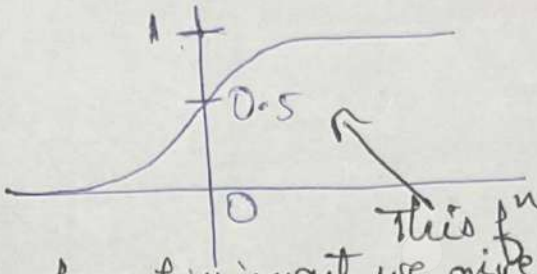
★ Perceptron-trick – A, B – decides the direction of the line
C – decides where the line sits.

} By comparing with random point in each epoch.

Adjust A, B, C again & again till the line seperates the data correctly.

Logistic Reg (Sigmoid f^n)

→ $\sigma(z) = \frac{1}{1+e^{-z}}$



→ Sigmoid converts the linear output of logistic Regression into probability b/w 0 & 1.

→ After this we apply a decision rule, if prob ≥ 0.5, class = 1
if prob < 0.5, class = 0.

→ We replaced the step f^n in perceptron trick with sigmoid f^n as perceptron had hard output of '0' or '1'.

Logistic Regression {Loss f^n / Max Likelihood}

→ With perceptron & sigmoid still not the best output.

crux- In ML we look for a loss f^n / Error f^n. The diff of predicted & actual by a formula. After we know the formula we find the minima & the coefficient over there are the actual good solution

Steps :-

1) Start with input data
2) Assume a linear eq^n.
3) Convert to probability using **Sigmoid**.
4) Calculate loss (log loss)
5) Update weights / coefficients using **gradient descent**
6) Stop training if max iteration reached or loss stops decreasing.
7) Make final prediction, $P \geq 0.5 = 1$ else $P < 0.5 = 0$.

Note - Sigmoid alone didn't tell us the best weight thus we used gradient descent to find it.

Classification Metrics [Accuracy & Confusion Matrix]

1) Accuracy

→ $$\text{Accuracy} = \frac{\text{No. of correct prediction}}{\text{Total predictions}}$$

] In multiclass classification also same formula & logic is applicable.

How much accuracy is good?
Depends on the problem, eg for self driving car high stake need 100% or in case of demand forecasting 80% also fine.

→ Accuracy = 90% so error = 10%
It does not tell which type of error.] Accuracy don't tell nature of mistake.

2) Confusion Matrix

→ Also tells the type of error.

→ True Positive: Predicted-1, Actual-1] Correct prediction

→ False Negative: Predicted-0, Actual-1] Wrong Prediction

→ False Positive: Predicted-1, Actual-0] Wrong Prediction

→ True Negative: Predicted-0, Actual-0] Correct prediction

	Prediction 1	Prediction 0
Actual 1	True Positive (TP)	False Negative (FN) Type 2 error
Actual 0	False Positive (FP) Type 1 error	True Negative (TN)

Hack to learn:-
eg True Positive
From Actual matches the predicted
If prediction '1' means positive

→ So the Accuracy = $\frac{\text{No. of correct Pred}}{\text{Total Pred}}$

→

$$\text{Acc} = \frac{TP+TN}{TP+TN+FP+FN}$$

Note
- False Positive = Type 1 error
- False Negative = Type 2 error

→ In case of multiclass classification also same formula for accuracy, the sum of left diagonal.

→ Why and why is accuracy misleading:-
In the case of imbalanced dataset eg → Y = 900, → N = 100.

Precision, Recall & F1 score

→ These help when Accuracy doesn't help.

• Precision

① ✓

	Sent to spam	Not sent to spam
Spam	100	170 FN
Not spam	300 FP	700

② ✓

	Sent to spam	Not sent to spam
Spam	100	190 FN
Not spam	10 FP	700

Both have same accuracy, which is better?

FP for ① > FP for ② which is crucial here so $P_B > P_A$

** as we want to see lowest error in Not spam being marked as spam

→ Thus, we chose model B (Proportion of predicted positive are actually positive)

$$\text{Precision} = \frac{TP}{TP + FP}$$

→ If model say YES how often is it correct.

• Recall

① ✓

	Detected cancer	Not Detected
Has cancer	1000	200 FN
No cancer	800 FP	8000

② ✗

	Detected cancer	Not Detected
Has Cancer	1000	500 FN
No cancer	500 FP	8000

Here also accuracy are same, so we can chose by Type1 or Type 2 error.

Here False Negative matters over False positive

(What proportion of actual positive is correctly classified)

$$\text{Recall} = \frac{TP}{TP + FN}$$

→ Out of all actual YES cases how many did we catch.

So we select model 1 here since $R_A > R_B$

• F1 score

→ Tells us how good the model is when both Type 1 & Type 2 error matters.

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

** → F_1 is high only when both Precision & recall are high & vice versa is true.

• Multiclass precision & recall

→ Here Precision & recall are calculated per class

→ Precision =

$$\frac{P_{Dog} + P_{cat} + P_{Rabbit}}{3}$$

Predicted

Actual	Dog	Cat	Rabbit	Total	Recall
Dog	25	5	10	40	0.62
Cat	0	30	4	34	0.88
Rabbit	4	10	20	34	0.58
Total Actual	29	45	34		
P	0.86	0.66	0.58		

→ Recall =

$$\frac{R_{Dog} + R_{cat} + R_{Rabbit}}{3}$$

→ F1 Score =

$$\frac{F_1 Dog + F_1 cat + F_1 Rabbit}{3}$$

Softmax Regression Multinomial Logistic Regression

→ Till now we studied Logistic Regression in Binary setup {0, 1}

→ Now we have more than 2, eg → Yes, No, Optout. classes in output.

→ For each class model computes a score z_1, z_2, z_3.

→ Softmax converts these score to probab b/w 0 & 1 → [0.2, 0.7, 0.2]

→ Pick the class with highest probab. for each point

→ For each point model computes probabilities of all (3) classes & picks the one class with highest probability.

Polynomial Logistic Regression

→ Same like Polynomial linear Regression

→ The algorithm is still logistic only the input features are transformed using polynomial degree to capture non-linear patterns.

→ One feature:-
- Degree = 1 → X (Normal Logistic Regression)
- Degree = 2 → X, X^2
- Degree = 3 → X, X^2, X^3

} we need to find optimal degree. Trial & error (Validation)

→ Two features:-
- Degree = 2 → $X_1, X_2, X_1^2, X_2^2, X_1 \cdot X_2$

→ higher degree means more features so we typically keep the degree small or use regularization.

Logistic Regression Hyperparameters [Imp only]

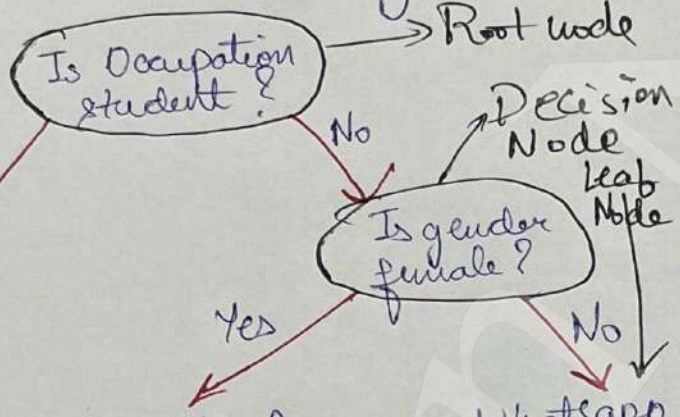
→ 'C' = Regularization strength, controls how much reg is applied
- C = 1/λ, smaller 'c' = stronger regularization = simpler model.
and vice versa, (default = 1.0)

→ 'penalty' = Type of regularization [l_1, l_2, elasticnet, none] (default = l_2)

→ 'max_iter' = max iteration for convergence (default = 100.)

Decision Trees

→ Decision tree is nothing but an if-else statement.




```
if occupation == student
    print(PUBG)
Else
    if gender == female
        print(Github)
    else
        print(Whatsapp)
```

→ Can work with both categorical & Numerical.
→ Disadvantage - overfits or perform poor on imbalanced data.
→ DT is a supervised technique used for classification & Regression.

Classification [Focus on class purity]

→ Target variable is categorical
→ Splitting criteria :-
1) Entropy check (Purity check)/Gini
2) Split on features
3) Calculate Information Gain
4) Chose Best split based on feature IG
5) Repeat until all nodes are pure

→ eg customer churn prediction

Regression [Focus on error minimization]

→ Target value is numerical
→ Splitting Criteria :-
1) Try splitting on each feature
2) Calculate error after each split
3) Chose the split with min error.
4) Repeat for child nodes.

(eg) - Predicting house prices.

Decision Tree Induction Extended

ID3 (Iterative Dichotomiser 3)
- Classification only
- Multiway tree structure
- Rare real world use
- Split Criteria (Entropy, IG)

CART (Classification & Regression Trees)
- Both classification & regression
- Strictly binary tree structure
- Very common (basis of Random forest)
- Split Criteria Gini (Classification), MSE/Variance (regression).

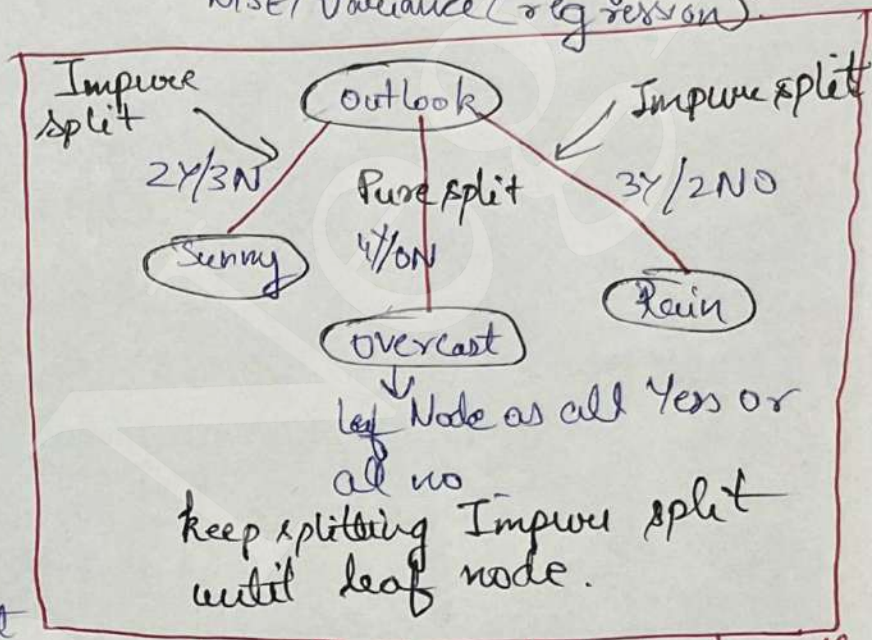
a) Entropy & Gini Index
- Techniques to check purity
- Entropy:

Range - [0, 1]
0 → pure, 1 → max impure

- ~~CART~~ Gini Index:

Range - [0, 0.5]
0 → pure, 0.5 → max impure

→ performs better on big dataset as Entropy formula has log it takes more time, Gini formula is simple math.



(b) Information Gain - Chose which feature to use for splitting
Whichever feature has higher IG, use that to split the tree.

Pre-Pruning
- → Applied during tree construction
- → Technique - max-depth etc etc
- → Used for bigger dataset

Main idea:-
Stop the trees early growth
Stops splitting during training

Post Pruning.
- → Applied after tree construction
- → Technique validation error, reduced error pruning
- → Used for small dataset

Main idea:-
Grow full tree & then remove weak branch.
Pruning happens after training
Remove branches that don't Validation performance

Note: Both reduce overfitting as tree algo are prone to overfitting

Nota

- Entropy – how mix or impure the data is in a node.
  High entropy means unordered set
  eg Hard to predict next card in unordered deck
  Low entropy means ordered set
  eg Easy to predict next card in ordered deck

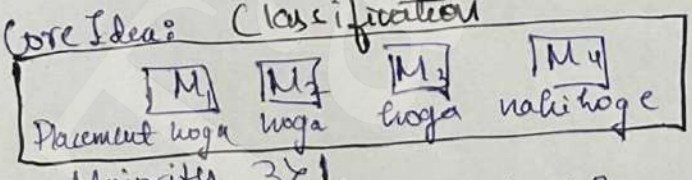
} Dec. Tree chose the split with largest entropy reduction

- Information gain – It measures how good a feature is in reducing the entropy
- Gini Index – Conceptually same like entropy at node level. Only mathematical formulas are different.
  Slightly faster than entropy.

- Controlling overfitting:-
  "max_depth" hyperparameter = Limits tree depth
  max_depth = 1 = Underfitting, max_depth = none [fully grown D.T]
- "max_features" = Handles dimensionality(higher) & avoid overfitting
  Controls how many features the tree considers at each split.

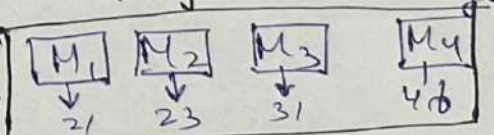
Ensemble learning

→ Single D.T is prone to overfitting, thus ensemble is more accurate.
→ Ensemble learning means combining multiple M.L models.
→ Wisdom of the Crowd – Means the crowd knows better
  eg – we rely on the crowd movie rating for credibility.
  ] Backbone of Ensemble
→ Each model (base model) should be different – different kind.
→ Core Idea: Classification



⇒ Placement hoga [Majority Count]

Regression eg: Predicting age



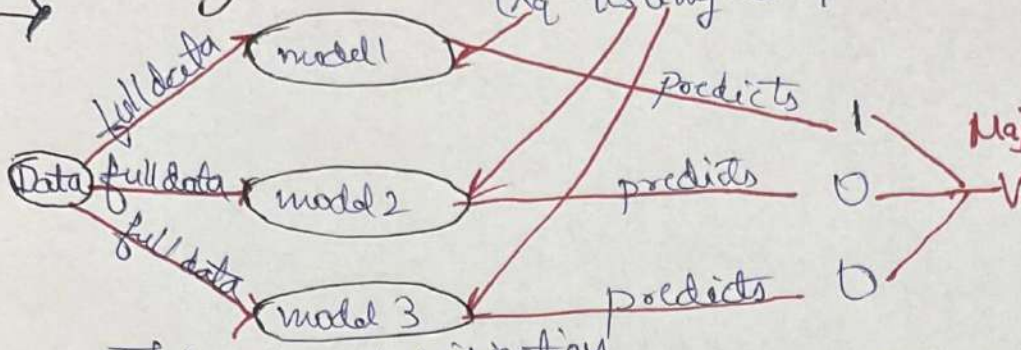
Prediction = Average of all the four models.

→ We use Ensemble learning mostly every time in practice [Good Results]

Types of Ensemble learning

- Voting Ensemble
- Bagging
  - Random forest
- Boosting
  - Adaboosting
  - Gradient boosting
  - Xg boost
- Stacking

1) Voting Ensemble



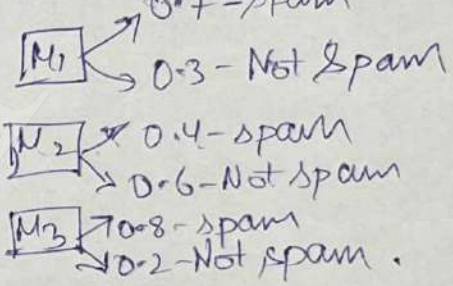
This is classification
In case of Reg. we take mean of 3 predictions

Assumptions:-
→ All models are different
→ All model accuracy > 50%.

Voting Ensemble ~~Classification~~ Types

- **Hard voting**
  Classification - Class with most votes win.
  Regression - We take the average of Predictions

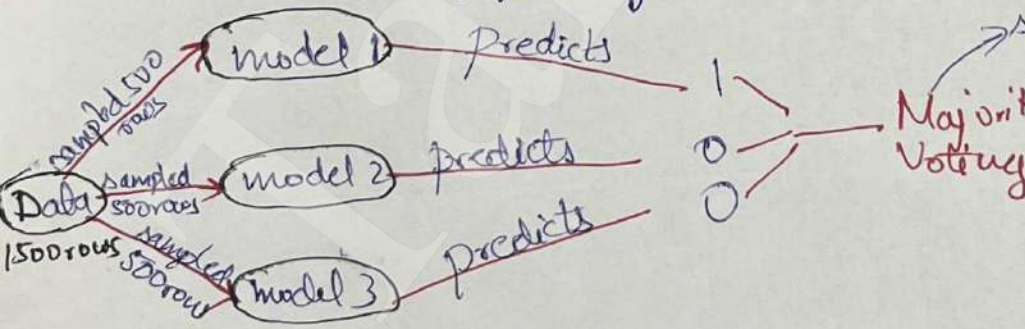
- **Soft voting**
  Classification - Each model predicts class probability instead of label & probability for each class are averaged



Spam Prob = 0.7 + 0.4 + 0.8 / 3 = 0.63
Not spam Prob = 0.3 + 0.6 + 0.2 / 3 = 0.37

2) Bagging

→ We chose two Bias, high Variance models {overfitting} ** {LBHV} ** Since data is sampled for each model
→ Thus we use bagging, resulting model is {LBLV}



Assumption:-
→ All models are same
→ Models are LBHV.

So Bagging reduces the variance as impact is reduced bec of data sampling to each model.

Bootstrapping (Sampling/training models on different dataset)

Aggregation

Random Forest

→ Supervised ML algo works for both regression & classification
→ Not much tuning needed in the algo.
→ It is an ensemble technique [Collection of Decision Tree]
→ It is a Bagging technique, intuition is same here.
→ For data sampling, for each $DT_1, DT_2 \cdots$?-

- row sampling or
- column sampling or
- combination

} for each DT with replacement or without replacement. ***

→ Why Random forest perform so well?

If converts LBHV ⟶ LBLV.
Noisy points impact is shared because of data sampling
Thus it solves Bias Variance Tradeoff issue.

→ Random forest Vs Bagging

- In bagging base estimator could be (DT, SVM, KNN etc)
  In RF the base estimator is always DT.
- In bagging & RF for data sampling if we do column sampling:- **

	Bagging	RF
Tree lvl Sampling	At each node sampling is done from ~~remaining~~ same selected columns for respective DT/SVM.	At each node sampling is done from full set of columns for respective DT.
	Thus more randomness	Node lvl sampling

- Thus many times RF is better than Bagging.

→ Random forest hyperparameters.

1) Bootstrap - True (Standard RF) / False (Entire dataset)
2) n-estimators - no. of trees in forest.
3) max-depth - Max. depth of tree
4) min_sample_split - min sample required to split a node.
5) Max-features - features considered at each split
6) min_sample_leaf - min data point that must remain in leaf.

→ Both classifier & Regressor have respective parameters, we chose the best by hyperparameter tuning → Grid Search CV / Randomized Search CV

Grid Search CV

tries all possible combinations of hyperparameter you give

Give parameter grid

↓

Create all possible combinations

↓

Train model for each combination

↓

Apply Cross-Validation (CV)

↓

Select best parameter set

Randomized Search CV

Randomly samples combination from the parameter space/you give

-Control randomness = 'n_iter'

Give parameter grid

↓

Randomly pick N combinations

↓

Train model for each combination

↓

Apply Cross Validation (CV)

↓

Select best param.

→ **Notes**

- Training data - Data used to teach model
- Validation data - Part of training data

Acts as a practice test before exam

Roughly 10% of ~~Validation~~ data = Validation

90% training data

We train several model & evaluate best on the validation set

We don't use test data to select model as indirectly we would train on the test data. **

As model has not seen validation data we hyperparameter tune using it not using the train data **

ML-Flow

Raw data

↓

Train-Test Split (80-20)

↓

Training data

↓

Cross Validation (inside training)

↓

Hyperparam tuning

↓

Best model selected

↓

Final Eval using Test data

OOB Evaluation

(oob_score = True) hyperparameter → Thus Out Of Bag Evaluation

→ This can be used as validation set for the model.

→ As the model has never seen this data

→ While sampling data [few data ~ 37% data] is never sent to any model in Bagging.

with repetition

→ Thus no need of validation data.

→ Helps hyperparameter tuning without splitting the data.

Feature Importance

→ Means to calculate importance of every feature / column
→ It is a feature selection technique.
→ We can keep imp features & remove unnecessary.
→ Basically the feature importance score tells us how much each feature contributes to the prediction of target.

→ Techniques
- **Shap** – Measures exact contribution of each feature
  If I remove one feature how much prediction change tells importance
- **Decision tree** – The feature that reduces the impurity most is most imp.
- **Random forest** – How imp is one feature to one or more trees. Not just based on one tree.
  How much overall impurity it reduces.

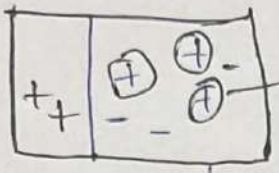
Note → Impurity = how mixed confused the data is in a node.
node has all same class, impurity = 0
" " different classes mixed, impurity > 0

---

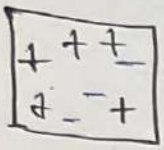
3) Boosting

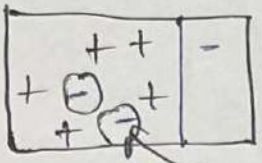
★ Adaboost

→ Weak learners – Ml model with accuracy just over 50%.
In adaboost we combine multiple weak learners together.
→ Decision stumps – Type of weak learner. DT with max_depth = 1.
In adaboost we use decision stump weak learner
→ Classification is on (-1, 1) not (0, 1) like other classification.
→ We boost the misclassified points before sending them to the next stage by upsampling.
→ Uses sequential learning
→ Each model tries to correct errors of the previous one.
→ Assigns weights to all data points with higher weightage to the misclassified points
→ This forces the next model to focus more on the mistakes

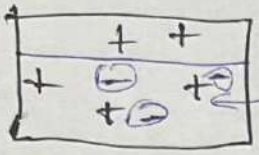
①  → These 3 points are misclassified so before sending to next we upsample these

α_1 – calculated based on performance

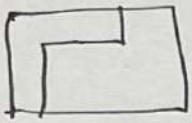


②  Now these 2 are misclassified we upsample these

α_2 – calculate based on performance

③  Again these 3 are misclassified

α_3 – Calculated based on performance

Now  → Final decision boundary, Result = + = Yes { Classification Logic
− = No.

→ Imp hyperparameters:-

- n_estimators = No of weak iteration, no. of models
- learning rate = control how much imp each model get in prediction
- random_state = controls randomness.

BAGGING VS BOOSTING

Recap – We need LBLV models, Regularization, bagging & boosting are 3 techniques.

Bagging

→ Uses LBHV models. eg- fully grow D.T

→ Does Sequential learning basically step by step for each model.

→ Weightage of base learner is same.
Each model have equal say

Boosting

→ Uses HBLV models, eg- shallow DT, max_depth = 1.

→ Does Parallel learning basically parallely training happens.

→ Weightage of base learner is different.
Better model gives a better say and Vice-Versa

K-means Clustering Algorithm.

→ Unsupervised learning & is a clustering algo not regression or classification

→ Each cluster has a centroid (center point)

→ We define the no. of clusters

→ Suppose we gave clusters = 3

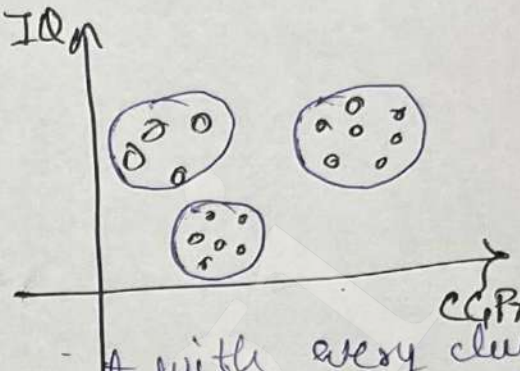
→ It randomly takes 3 point & then

→ calculate the Euclidian distance of each point with every cluster.

→ The point will be assigned to cluster that have lowest Euclidian distance

→ Now we have 3 cluster, we again find centroid of each cluster.

→ Finish condition - In current & last step the centroids didn't move.



→ How to know no. of clusters?

• We decide no. of algorithm not randomly but using elbow method (using WCSS Vs K number of clusters graph)

Gradient Boosting

→ Better than adaboost, enhanced version is XG Boost, this also use trees.

→ Both regression & classification model.

for Regression

n-estimators

Model 1 + Model 2 + Model 3 .... till ~~actual pred~~

iq	cgpa	package (lpa)	pred 1	res 1
90	8	3	4.8	-1.8
100	7	4	4.8	-0.8
110	6	8	4.8	3.2
120	9	6	4.8	1.2
80	5	5	4.8	-1.8

.... New model keeps adding

'X' | Target column (Y) Actuals | Output of model1 Always mean of regression | res1 = Actual - pred 2

Now for model 2:- ***

X is same but 'Y' = res1.

this is done till the residual is near 0 decided by n-estimators.

prediction = m1 + λm2 + λ3 ....

λ = learning rate, normally 0.1

For every new model we take recent residual as target.

→ In Gradient Boosting classification changes.

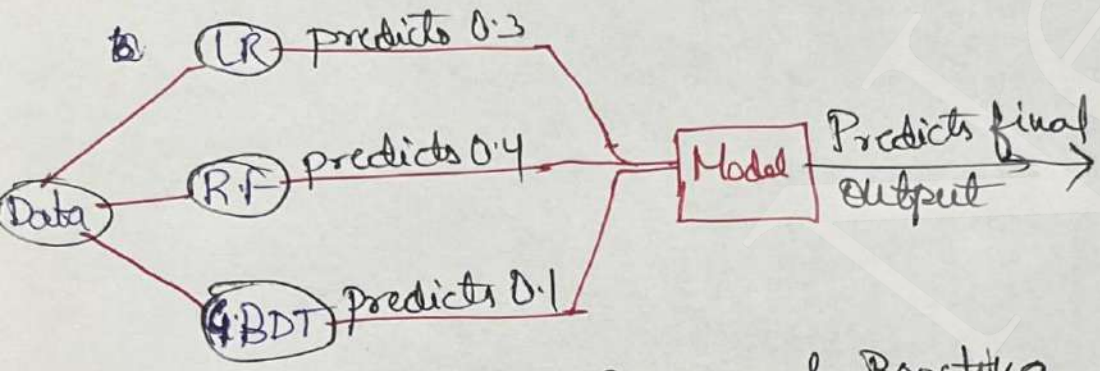
- the model one prediction is = $\log\left(\frac{\text{odd counts, basically '1' counts}}{\text{even counts, basically '0's counts}}\right)$
- Find trees instead of residuals we try to reduce the log error

---

4) ~~Stacking~~ ~~Blending~~ Ensemble & Blending

→ Extends the idea of Voting Ensemble.

→ The output of every model is input to our meta model.



→ How is it diff from Bagging & Boosting.

- Base models are different unlike bagging & boosting
- Output of the base models are again used for training directly.

→ Problem with stacking.

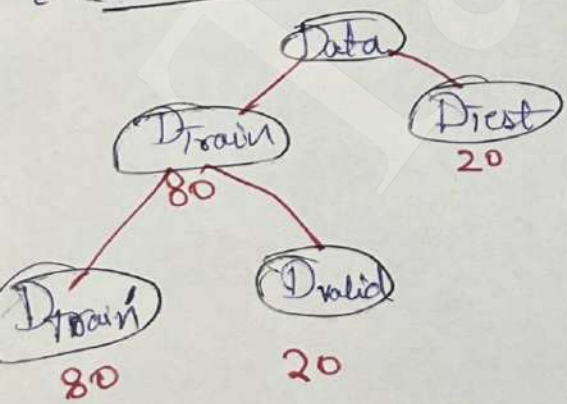
We train & predict using the same data so overfitting occurs.

Solution

Blending (Hold out method)

Stacking (Ki Bolte ka ise Bi) (K-fold method)

★ Blending



Steps:-

→ Train 3 base models on D_{Train} & pred with D_{Valid}

→ Pred. of 3 base + ~~original~~ target = new dataset

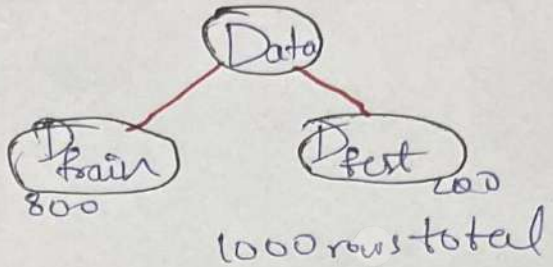
→ Train the Meta model with new dataset

→ Test the Meta model with D_{Test}

* K-folds (Stacking)

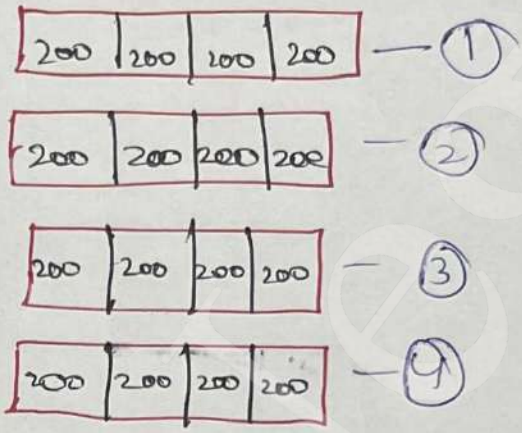
→ K-fold like cross validation

→ we decide a k, eg K=4



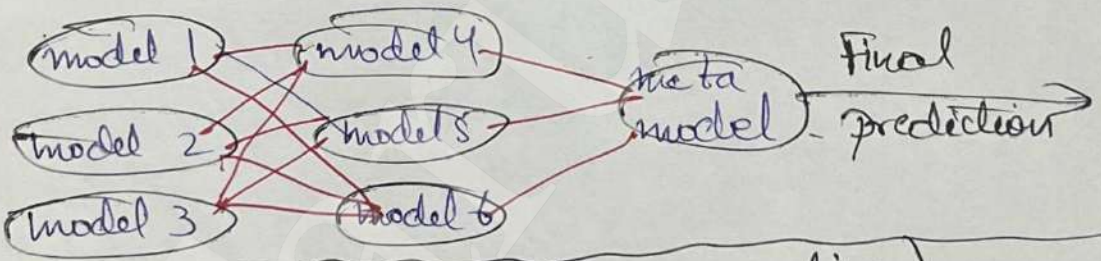
Step 1

- Base model 1 train 4 times from each ① ② ③ ④
  Pick 600 columns
  Remaining 200 for test
- After testing we have Prediction of all 800 rows of model 1



- Same steps for model 2, model 3.
- Now we have new dataset
- Use this dataset to train the metamodel.
- Retrain each Base model with Dtrain.

→ Multi-layer stacking



Hierarical Clustering (Agglomerative)

→ Problem with K-means clustering :-
For complex data like concentric circle, K means centroids fail.
as it relies a lot on Euclidian distance.

more clustering methods

- Hierarchical clustering
  - Agglomerative clustering — more used
  - Divisive Clustring
- Density Based Clustring (DBSCAN)

✱ Agglomerative

→ Bottom up clustering method

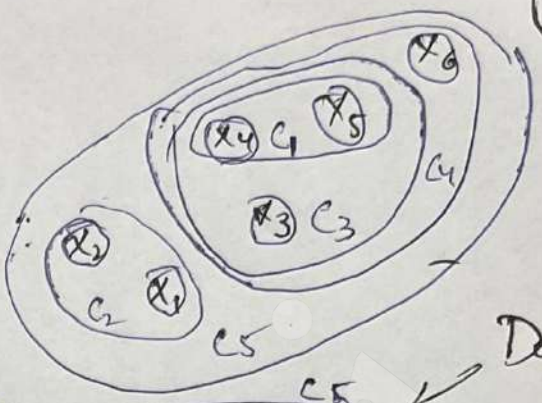
→ Start with assuming each point as a cluster

↓

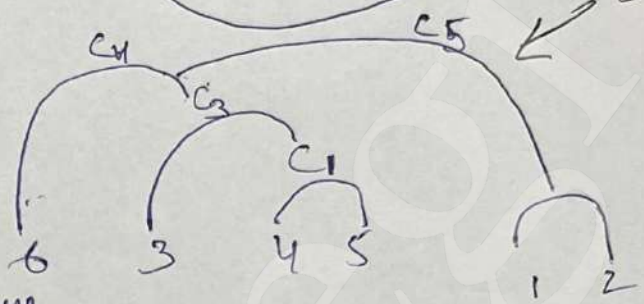
Repeatedly merge closest clusters

↓

Stop when desired cluster is reached or distance threshold is met



Dendogram



Note - Divise assume all points inside one single cluster
Works opp. of Agglomerative Top-down.

→ Types of Agglomerative clustering :- [linkage]

- The types are on the basis of how we calculate dist. b/w clusters.
- Single link(Min) =  finds min dist. b/w closest point.
- Complete link(Max) = finds max dist. b/w closest point
- Average = Mean of all points.
- Wared = We minimize the variance to get compact clusters.

→ How to find the number of clusters - Using Dendrogram Cut.

→ Benefits of hierarchical clustering :-

- Unlike K-means we don't have to chose K upfront can decide after seeing the dendogram
- Dendograms show us how clusters are formed
- Works with different distance/linkage method.

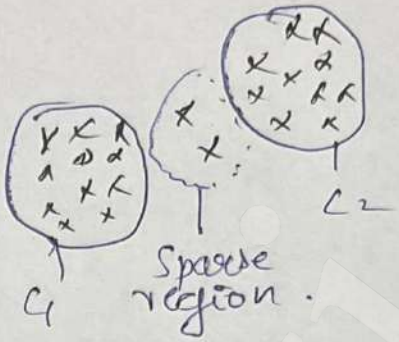
→ Limitation - Can't work on big datasets

→ Hyperparameters :-

- linkage - controls the cluster shape & merging behaviour.
- affinity - 'euclidian', 'manhattan', 'cosine' - Distance measure b/w 2 points.
- n_clusters - number of clusters to form
- distance_threshold - Stop merging when distance exceeds threshold.

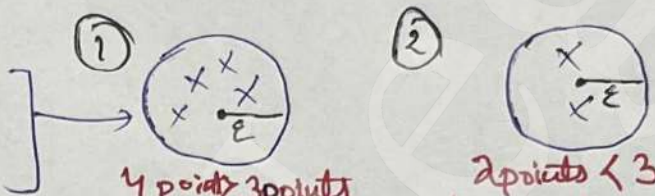
DBCAN**

→ flaws of K-means :-
- to define the clusters upfront
- very sensitive to outliers.
- K-means doesn't work with complex non-spherical data (because of centroid)



→ DBSCAN is a density based clustering we cluster with density b/w the datapoints with a help of sparse region

⇒ Imp hyperparameter :-
'epsilon' - radius of circle
'minpts' - points threshold in the circle
eg - 3



4 points > 3 points
Thus Dense region

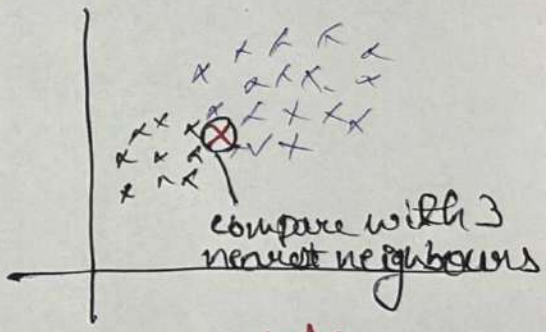
2 points < 3 points
& parse region

→ DBSCAN could shapes better but Hierarchical is more Interpretable.

K-Nearest Neighbour

⇒ Supervised, works both for classification & Regression

Philosophy - "You are the average of the 5 people you spend most time with"



→ K=3, (Suppose)
We compare with 3 nearest neighbours based on Euclidian distance & take majority of their prediction

→ Works with multiple dimensions.

→ How to chose 'K' = No fixed method [small K = overfitting, large K = underfitting] **

→ We chose best 'K' using cross validation

→ Limitation of KNN :-
- Slow prediction time - As calculate distance to all training points.

Thus inferencing time is expensive & takes more memory, thus expensive

Testing = testing how good the model is
Inferencing = using the model with real world data

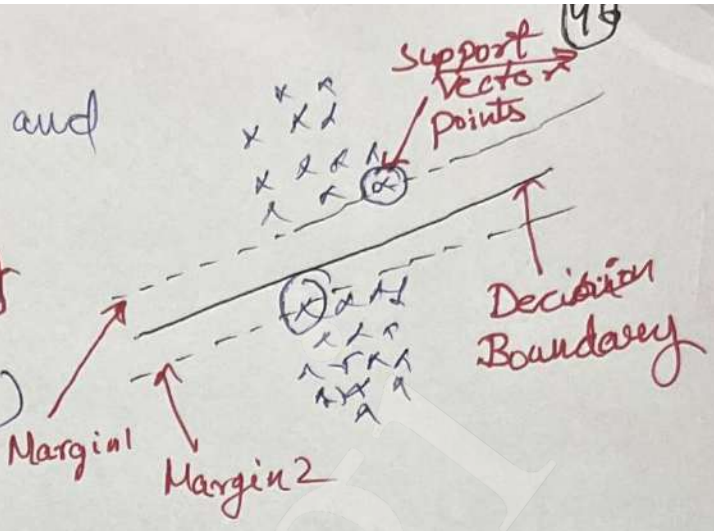
→ Decision Boundary - line or curve for 2D feature space.
Decision Surface - General term for seperation b/w classes in 3D or it' feature space

KNN forms decision surface at time of inferencing based on 'K'
Small 'K' - Irregular decision surface, Large K - smooth & stable.

Support Vector Machines

→ Supervised ML Algo used for Regression and classification

→ SVM finds the decision boundary with max. margin distance b/w classes



→ Line (2D), plane (3D) Hyperplane (higher-D)

→ Note - Margin = distance b/w boundary & closest point

→ SVM are robust to outliers, can also work with Non-linear data using Kernels

Kernel - Trick – Kernel lets SVM solve non-linear problems by implicitly mapping data into a higher-dimensional space where it becomes linearly separable. Most used (default)

Imp Kernels – Linear, Polynomial / RBF (Gaussian), Sigmoid.

→ Other hyperparameters :-

- 'C' (Regularization parameter) – control tradeoff b/w margin & classification error.

Small 'C' – Underfitting risk – Allows more mis-classification

Large 'C' – Overfitting risk – A large C forces the model to fit training data perfectly.

→ SVM limitation – slow on training large data & sensitive to Kernel/hyperparam

---

Naive Bayes Classifier

→ Supervised, used only for classification

→ Condition Probability = P(A|B) = P(Rain | Cloudy) = P to Rain given its cloudy

→ Mutually exclusive = Two events cannot happen at the same time.
$P(A \cap B) = 0$ (eg) coin toss → head or tail.

→ Independent event = Two events where one does not affect the other.
$P(A|B) = P(A)P(B)$, tossing one coin don't affect other

→ Bayes theorem → Given $(P) \neq 0$

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

→ Easy words - To update what we have already seen.

Here total sample space Now is rows with cloudy.

"Naive" assumption is it assumes all features are conditionally independent given the class

$P(A|B)$ = posterior (Updated belief) $P(A)$ = prior (What you believed before)

$P(B|A)$ = likelihood (How likely the evidence is) $P(B)$ = evidence (New info)

New belief = Old Belief × How well is the evidence support

Eg)

Boss	Venue	Outlook	Result
Won	Mumbai	Overcast	Won
Lost	Chennai	Sunny	Won
Won	Kolkata	Sunny	Lost

→ Find {Lost, Mumbai, Sunny} → Result = ?

If find P(W) & P(L), which is his result.

P(W | Lost ∩ MI ∩ Sunny) = Apply Bayes formula

P(L | Lost ∩ Mumbai ∩ Sunny) = " " "

⇒ P(W | Lost ∩ Mumbai ∩ Sunny) = 0.2 < P(W | Lost ∩ Mumbai ∩ Sunny) = 0.3

Thus we classify {Lost, Mumbai, Sunny} As 'Won'.

XGBOOST (extreme Gradient Boosting)

→ XGBOOST is library built on top of Gradient Boosting.

XGB → Performance ✓ Speed ✓ Flexibility ✓

1) ~~Performance~~ Flexibility
- Cross platform - Works on any O.S
- Multiple language support - Supports multiple programming language.
- Integration with other libraries & tool.
- Supports all kind of ML problems - Reg, Classification, Timeseries, Ranking, Huma
- ★ It can use any loss fn or custom loss fn like Gradient Boosting.

2) Speed [Why training time of XGB is less?]
- Parallel processing - Training individual Model in Boosting happen in parallel. The overall process is still sequential as it is boosting.
- Optimized data structure - Uses 'Colum Block' stores info in column wise not row wise.
- Cache Awareness - Uses cache memory efficiently [Cache memory is in CPU only]
- ★★ Out of Core Computing - Big data is broken in chunks & sequential model training
- Distributed Computing - Can train huge data & can train using different machine.
- GPU support - Processing can be done through GPU

3) Performance
- Regularized Learning objective - Loss fn by default has regularization term ★★
- Handling missing values - XGB handle sparsity itself using sparse aware split - finding

- Efficient Split finding (Weighted Quantile Sketch + Approximate Tree Learning) - finding the best feature to split a node with less memory computation
- Tree pruning - To reduce overfitting [Pre-Pruning, Post-Pruning]

Light GBM (Light Gradient Boosting Machine)

- Works faster than XGB on larger datasets
- XGB is handles overfitting better but -

Light GBM is a fast gradient boosting framework that uses histogram based split finding & leaf-wise tree growth to achieve high accuracy & efficiency on large dataset [sparse datasets]

XGB for Regression

→ Prediction of first model is Mean of target 'y'
→ for subsequent models Input 'x' is same but 'y' is residual or the error of the last model [We try to predict error]

CGPA	Package	Model1	residual1
6.7	4.5	7.3	-2.8
9.0	11.0	7.3	3.7
7.5	6.0	7.3	-1.3
5.0	8.0	7.3	0.7

Now, X & Y for the next D.T.

→ How does we decide where to split:

Using

$$\text{Similarity score} = \frac{(\text{Sum of residuals})^2}{\text{No. of residuals} + \lambda}$$

(λ → regularization parameter)

$$\text{SS of root} = \frac{(-2.8+3.7-1.3+0.7)^2}{4+\lambda} = 0.02 \text{ eg}$$

→ First we sort the X
5.0
6.7 } criteria 1 — 5.85 mean
7.5 } criteria 2 — 7.1 mean
9.0 — criteria 3 — 8.25 mean

criteria 1

Cgpa < 5.85

0.7 — $SS_L = 0.49$

-2.8, -1.3, 3.7 — $SS_R = 0.05$

Gain = $(SS_L + SS_R) - SS_{root}$
= 0.52

criteria 2

Cgpa < 7.2

Same gain calculation here

Gain = 5.06

criteria 3

Cgpa < 8.25

Same gain calculation here

Gain = 17.52 — max so we chose

→ So that's how we split tree based on SS

Prediction = $M_1 + lr * M_2 + lr + M_3$

with every new tree addition we reduce residual toward 0.

→ Splitting on single feature is done, how do we chose which feature to split on if we have more than 1 input feature. The model tries all possible split points & chose the feature-split pair that gives the max gain (loss reduction)

→ XGB classifier - Base like GBDT classifier works but tree is made differently with regularization term.

→ XGB hyperparameters (IMP only)

Tree structure
- max_depth - How complex each tree is
- min_child_weight - Min samples in leaf
- gamma - min gain to split.

Boosting control
- n_estimator → No. of trees
- learning rate (eta) - Step size (how much each tree contributes)

regularization
- reg_lambda (λ) - L_2 reg - shrink leaf weights.
- reg_alpha (α) - L_1 reg - Forces weak split to '0'.

Note: XGB handle sparsity by learning optimal default direction for missing values at each split & efficiently skipping 0 entries during training

---

Imbalanced Data in Machine Learning

→ In ML we often deal with Imbalanced data eg:-

1000 mail → Spam - 50, Not spam - 950 (so 5% vs 10%)

The model will become bias with 95% accuracy but low recall.

Techniques to handle Imbalanced Data.

Undersampling
- Spam = 50
- non-spam = 950
- we remove 900 rows
- Now, non-spam = 50
- Both spam and Non-spam are same size after this

Oversampling
- spam = 50
- non-spam = 950
- we duplicate 900 rows
- Now, spam = 900
- Both spam and Non-spam are same size after this.

SMOTE (Synthetic Minority Oversampling)
- Does Oversampling not by duplicating but by Interpolation
- eg) create a new spam b/w 2 spams.
- Rest same as Oversampling

Ensemble methods
- Train many model
- Each model sees spam = 50
- But different subset of non-spam
- Predictions become more stable

Cost sensitive learning.
- Tweak learning of model to handle imbalance data
- eg) giving more weight to minority (spam)
- So if minimize error at spam. 50 sample behave like 500

Hyperparameter Tuning using Optuna

- optuna is automatic way to search for the best hyperparameter by learning from past trials.

Grid Search — Slow, tries all combos

Random Search — Waste many trials

Optuna — Learns while searching by remembering which values gave good results & the focusing future searches around those values.

Pick hyperparameters → Train Model → Measure the performance → Learns from result → Try better values next time

CORE IDEA

ROC Curve in Machine Learning

→ Used for threshold selection for classification (binary) problems.

(eg)

ID	CGPA	Placed	prediction
7	70	1	0
8	80	0	0
9	90	1	1

→ the classification models like logistic Reg don't predict (0 or 1) they predict probability values eg 0.45, 0.39, 0.61

We decide a threshold to convert these probabilities to prediction **

eg 0.5 1, 0 else 1.

Actual \ Prediction	1	0
1	TP	FN
0	FP	TN

Confusion Matrix

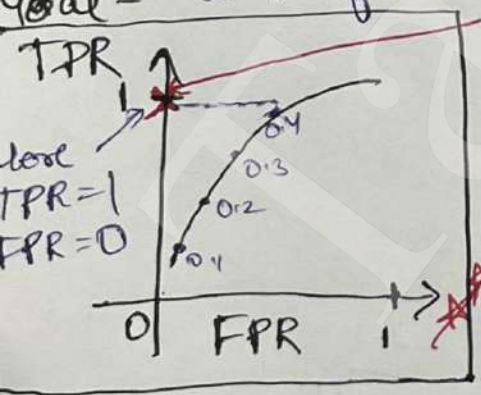
$TPR = \frac{TP}{TP+FN}$ → Also called recall (How many actual positives were correctly classified)

Goal is to maximize TPR while threshold selection.

$FPR = \frac{FP}{FP+TN}$ (How many negative samples are wrongly predicted as positive)

Goal is to minimize FPR while threshold selection.

Overall Goal - Is to find a threshold which is closest to [TPR=1 & FPR=0]



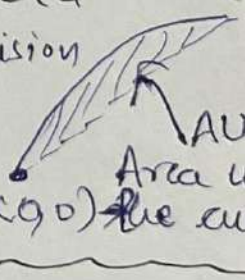
While testing we try diff values of threshold like (0.5, 0.6, 0.7), plot & chox the closest to this point

eg 0.4 here would be our threshold **

Now AUC-ROC is used for model comparision

b/w (0,0) to (1,1)

AUC = 1, Max, Model is good correctly classifies all 1's & 0's, 0 is poor model.



"IT'S NEVER TOO LATE"

- TARUN NEGI